

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to

determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Ashira Bridget Susan S

Experiment Questions:-

Experiment 1

Logical Address to Physical Address using Base and Limit Registers

Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

C Program

```
#include <stdio.h>

int main()
{
    int base = 500;
    int limit = 300;
    int logical, physical;
    printf("Enter Logical Address: ");
    scanf("%d", &logical);
    if(logical < limit)
    {
        physical = base + logical;
        printf("Valid Address\n");
        printf("Physical Address = %d", physical);
    }
    else
    {
        printf("Invalid Address");
    }
    return 0;
}
```

}

Explanation

- The program stores base and limit register values.
- It checks whether the logical address is within the limit.
- If valid, it adds the base value to obtain the physical address; otherwise, it reports an invalid address.

Pseudocode

Start

Set Base = 500

Set Limit = 300

Read Logical Address

If Logical Address < Limit

 Physical Address = Base + Logical Address

 Display Physical Address

Else

 Display Invalid Address

Stop

Sample Input

Enter Logical Address: 120

Sample Output

Valid Address

Physical Address = 620

main.c	   Run	Output
<pre>1 // Experiment 1 2 // Logical Address to Physical Address using Base and Limit Registers 3 4 #include <stdio.h> 5 6 int main() 7 { 8 int base = 500; 9 int limit = 300; 10 int logical, physical; 11 12 printf("Enter Logical Address: "); 13 scanf("%d", &logical); 14 15 if(logical < limit) 16 { 17 physical = base + logical; 18 printf("Valid Address\n"); 19 printf("Physical Address = %d", physical); 20 } 21 else 22 { 23 printf("Invalid Address"); 24 } 25 26 return 0; 27 }</pre>		<pre>Enter Logical Address: 120 Valid Address Physical Address = 620 === Code Execution Successful ===</pre>

Experiment 2

Paging Memory Management

Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

C Program

```
#include <stdio.h>

int main()
{
    int pageTable[4] = {5, 8, 2, 6};
    int page, offset, frame, physical;
    printf("Enter Page Number: ");
    scanf("%d", &page);
    printf("Enter Offset: ");
    scanf("%d", &offset);
    frame = pageTable[page];
    physical = frame * 100 + offset;
    printf("Frame Number = %d\n", frame);
    printf("Physical Address = %d", physical);
    return 0;
}
```

Explanation

- A page table maps page numbers to frame numbers.
- The user enters a page number and offset.
- The program finds the corresponding frame and calculates the physical address using the frame number and offset.

Pseudocode

Start

Create Page Table

Read Page Number

Read Offset

Find Frame Number from Page Table

Physical Address = Frame $\times$ 100 + Offset

Display Frame Number

Display Physical Address

Stop

Sample Input

Enter Page Number: 2

Enter Offset: 45

Sample Output

Frame Number = 2

Physical Address = 245

```
Output Clear  
Enter Page Number: 2  
Enter Offset: 45  
Frame Number = 2  
Physical Address = 245  
  
=== Code Execution Successful ===
```

Experiment 3

Page Table Simulation

Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

C Program

```
#include <stdio.h>

int main()
{
    int pageTable[5];
    int i, page, offset, frame;
    printf("Enter Frame Numbers:\n");
    for(i=0;i<5;i++)
    {
        printf("Page %d -> ", i);
        scanf("%d",&pageTable[i]);
    }
    printf("Enter Page Number: ");
    scanf("%d",&page);
    printf("Enter Offset: ");
    scanf("%d",&offset);
    frame = pageTable[page];
    printf("Frame Number = %d\n", frame);
    printf("Physical Address = %d", frame*100+offset);
    return 0;
}
```

Explanation

- The user first enters the page table entries.
- Then the program accepts a page number and offset.
- It translates the page number into a frame number and computes the physical address.

Pseudocode

Start

Input Frame Number for each Page

Read Page Number

Read Offset

Frame = Page Table[Page Number]

Physical Address = Frame × 100 + Offset

Display Frame

Display Physical Address

Stop

Sample Input

Page 0 -> 3

Page 1 -> 6

Page 2 -> 1

Page 3 -> 8

Page 4 -> 5

Enter Page Number: 3

Enter Offset: 20

Sample Output

Frame Number = 8

Physical Address = 820

Output

Enter Frame Numbers:

Page 0 -> 3

Page 1 -> 6

Page 2 -> 1

Page 3 -> 8

Page 4 -> 5

Enter Page Number: 3

Enter Offset: 20

Frame Number = 8

Physical Address = 820

=== Code Execution Successful ===

Experiment 4

Translation Lookaside Buffer (TLB)

Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

C Program

```
#include <stdio.h>

int main()
{
    float memory = 100;
    float tlb = 20;
    float hitRatio;
    float withoutTLB, withTLB;
    printf("Enter TLB Hit Ratio (0 to 1): ");
    scanf("%f",&hitRatio);
    withoutTLB = 2 * memory;
    withTLB = hitRatio * (tlb + memory) +
        (1-hitRatio) * (tlb + 2*memory);
    printf("Without TLB = %.2f ns\n", withoutTLB);
    printf("With TLB = %.2f ns", withTLB);
    return 0;
}
```

Explanation

- The program calculates memory access time with and without a TLB.
- It uses the TLB hit ratio to compute the effective memory access time.
- The result shows how a TLB improves performance by reducing average access time.

Pseudocode

Start

Set Memory Access Time = 100

Set TLB Access Time = 20

Read Hit Ratio

Without TLB = $2 \times \text{Memory Time}$

With TLB =

$(\text{Hit Ratio} \times (\text{TLB} + \text{Memory}))$

+

$(\text{Miss Ratio} \times (\text{TLB} + 2 \times \text{Memory}))$

Display Both Times

Stop

Sample Input

Enter TLB Hit Ratio: 0.8

Sample Output

Without TLB = 200.00 ns

With TLB = 140.00 ns

main.c

Share

Run

```
2 // Translation Lookaside Buffer (TLB)
3
4 #include <stdio.h>
5
6 int main()
7 {
8     float memory = 100;
9     float tlb = 20;
10    float hitRatio;
11    float withoutTLB, withTLB;
12
13    printf("Enter TLB Hit Ratio (0 to 1): ");
14    scanf("%f",&hitRatio);
15
16    withoutTLB = 2 * memory;
17
18    withTLB = hitRatio * (tlb + memory) +
19             (1-hitRatio) * (tlb + 2*memory);
20
21    printf("Without TLB = %.2f ns\n", withoutTLB);
22    printf("With TLB = %.2f ns", withTLB);
23
24    return 0;
25 }
```

Output

Enter TLB Hit Ratio (0 to 1): 0.8
Without TLB = 200.00 ns
With TLB = 140.00 ns

=== Code Execution Successful ===

Experiment 5

Memory Protection using Base and Limit Registers

Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

C Program

```
#include <stdio.h>

int main()
{
    int base = 1000;
    int limit = 400;
    int logical;
    printf("Enter Logical Address: ");
    scanf("%d",&logical);
    if(logical >= 0 && logical < limit)
    {
        printf("Access Allowed\n");
        printf("Physical Address = %d", base + logical);
    }
    else
    {
        printf("Access Denied");
    }
    return 0;
}
```

Explanation

- The program uses base and limit registers to enforce memory protection.
- It verifies whether the logical address is within the allowed range.
- If valid, access is allowed and the physical address is calculated; otherwise, access is denied.

Pseudocode

Start

Set Base Register

Set Limit Register

Read Logical Address

If Address is between 0 and Limit

 Allow Access

 Calculate Physical Address

Else

 Deny Access

Stop

Sample Input

Enter Logical Address: 350

Sample Output

Access Allowed

Physical Address = 1350

main.c

Share

Run

```
3
4 #include <stdio.h>
5
6 int main()
7 {
8     int base = 1000;
9     int limit = 400;
10    int logical;
11
12    printf("Enter Logical Address: ");
13    scanf("%d",&logical);
14
15    if(logical >= 0 && logical < limit)
16    {
17        printf("Access Allowed\n");
18        printf("Physical Address = %d", base + logical);
19    }
20    else
21    {
22        printf("Access Denied");
23    }
24
25    return 0;
26 }
```

Output

Enter Logical Address: 350
Access Allowed
Physical Address = 1350

=== Code Execution Successful ===

Experiment 6

Dynamic Storage Allocation (First Fit, Best Fit, Worst Fit)

Develop a program to simulate dynamic storage allocation using First Fit, Best Fit, and Worst Fit strategies. The program should accept the sizes of available memory blocks and process requests, allocate memory using each strategy, and display the allocated blocks and remaining unused memory.

C Program

```
#include <stdio.h>

int main()
{
    int block[5] = {100, 500, 200, 300, 600};
    int process[3] = {212, 417, 112};
    int i, j;
    // First Fit
    printf("First Fit\n");
    int temp1[5] = {100,500,200,300,600};
    for(i=0;i<3;i++)
    {
        for(j=0;j<5;j++)
        {
            if(temp1[j] >= process[i])
            {
                printf("Process %d -> Block %d\n", i+1, j+1);
                temp1[j] -= process[i];
                break;
            }
        }
    }
}
```

```

// Best Fit

printf("\nBest Fit\n");

int temp2[5] = {100,500,200,300,600};

for(i=0;i<3;i++)

{
    int best = -1;

    for(j=0;j<5;j++)

    {
        if(temp2[j] >= process[i])

        {
            if(best==-1 || temp2[j] < temp2[best])

            best = j;

        }

    }

    if(best!=-1)

    {

        printf("Process %d -> Block %d\n", i+1, best+1);

        temp2[best] -= process[i];

    }

}

```

```

// Worst Fit

printf("\nWorst Fit\n");

int temp3[5] = {100,500,200,300,600};

for(i=0;i<3;i++)

{

    int worst = -1;

    for(j=0;j<5;j++)

    {

```

```

    if(temp3[j] >= process[i])
    {
        if(worst==-1 || temp3[j] > temp3[worst])
            worst = j;
    }
}
if(worst!=-1)
{
    printf("Process %d -> Block %d\n", i+1, worst+1);
    temp3[worst] -= process[i];
}
}
return 0;
}

```

Explanation

- Memory block sizes and process sizes are stored in arrays.
- **First Fit** allocates the first block that is large enough.
- **Best Fit** allocates the smallest suitable block.
- **Worst Fit** allocates the largest available block.
- After allocation, the remaining block size is reduced.

Pseudocode

Start

Store block sizes and process sizes

Perform First Fit allocation

Perform Best Fit allocation

Perform Worst Fit allocation

Display allocations

Stop

Sample Output

First Fit

Process 1 -> Block 2

Process 2 -> Block 5

Process 3 -> Block 2

Best Fit

Process 1 -> Block 4

Process 2 -> Block 2

Process 3 -> Block 3

Worst Fit

Process 1 -> Block 5

Process 2 -> Block 2

Process 3 -> Block 5

Output

▲ First Fit

Process 1 -> Block 2

Process 2 -> Block 5

Process 3 -> Block 2

Best Fit

Process 1 -> Block 4

Process 2 -> Block 2

Process 3 -> Block 3

Worst Fit

Process 1 -> Block 5

Process 2 -> Block 2

Process 3 -> Block 5

=== Code Execution Successful ===

Experiment 7

Swapping

Write a program to simulate swapping between main memory and secondary storage. The program should accept the memory size, process sizes, and process arrival/removal requests, and display which processes are swapped in and swapped out at each stage.

C Program

```
#include <stdio.h>

int main()
{
    int memory = 500;
    int used = 0;
    int p1 = 200;
    int p2 = 150;
    int p3 = 250;
    used += p1;
    printf("P1 Swapped In\n");
    used += p2;
    printf("P2 Swapped In\n");
    if(used + p3 > memory)
    {
        printf("P1 Swapped Out\n");
        used -= p1;
    }
    used += p3;
    printf("P3 Swapped In\n");
    return 0;
}
```

Explanation

- Total memory is fixed.
- P1 and P2 are loaded first.
- Before loading P3, memory is checked.
- Since memory is insufficient, P1 is swapped out.
- Then P3 is loaded.

Pseudocode

Start

Load P1

Load P2

If memory is full

 Remove old process

Load P3

Display swapping

Stop

Sample Output

P1 Swapped In

P2 Swapped In

P1 Swapped Out

P3 Swapped In

Output

P1 Swapped In
P2 Swapped In
P1 Swapped Out
P3 Swapped In

=== Code Execution Successful ===

Experiment 8

Segmentation

Implement a program to simulate segmentation memory management. The program should accept segment numbers and offsets along with a segment table containing base and limit values. It should validate the offset and calculate the corresponding physical address for valid memory references.

C Program

```
#include <stdio.h>

int main()
{
    int base[3]={1000,2000,3000};
    int limit[3]={400,300,500};
    int seg, offset;
    printf("Enter Segment Number: ");
    scanf("%d",&seg);
    printf("Enter Offset: ");
    scanf("%d",&offset);
    if(offset<limit[seg])
        printf("Physical Address = %d",base[seg]+offset);
    else
        printf("Invalid Address");
    return 0;
}
```

Explanation

- Each segment has a **base** and **limit**.
- User enters a segment number and offset.
- Offset is checked against the segment limit.

- If valid, physical address = base + offset.

Pseudocode

Start

Store Base and Limit

Read Segment Number

Read Offset

If Offset < Limit

 Physical Address = Base + Offset

Else

 Invalid Address

Stop

Sample Input

Enter Segment Number: 1

Enter Offset: 150

Sample Output

Physical Address = 2150

Output

```
Enter Segment Number: 1
```

```
Enter Offset: 150
```

```
Physical Address = 2150
```

```
=== Code Execution Successful ===
```

Experiment 9

Segmentation with Paging

Develop a program to demonstrate segmentation with paging. The program should accept a logical address consisting of segment number, page number, and offset, use the segment table and page table for translation, and display the final physical address or an appropriate error message.

C Program

```
#include <stdio.h>

int main()
{
    int pageTable[2][3]={
        {2,4,1},
        {5,6,3}
    };

    int seg,page,offset;
    printf("Enter Segment: ");
    scanf("%d",&seg);
    printf("Enter Page: ");
    scanf("%d",&page);
    printf("Enter Offset: ");
    scanf("%d",&offset);

    int frame=pageTable[seg][page];

    printf("Frame = %d\n",frame);
    printf("Physical Address = %d",frame*100+offset);

    return 0;
}
```

Explanation

- A page table is stored for every segment.
- User enters segment, page and offset.
- Program finds the corresponding frame.
- $\text{Physical address} = \text{frame} \times \text{page size} + \text{offset}.$

Pseudocode

Start

Create Page Table

Read Segment

Read Page

Read Offset

Find Frame

$\text{Physical Address} = \text{Frame} \times 100 + \text{Offset}$

Display Result

Stop

Sample Input

Enter Segment: 1

Enter Page: 2

Enter Offset: 35

Sample Output

Frame = 3

Physical Address = 335

Output

```
▲ Enter Segment: 1
Enter Page: 2
Enter Offset: 35
Frame = 3
Physical Address = 335

=== Code Execution Successful ===
```

Experiment 10

Demand Paging

Write a program to simulate virtual memory using demand paging. The program should accept a sequence of page references and the number of available frames, identify page faults, load required pages into memory, and display the contents of each frame after every reference.

C Program

```
#include <stdio.h>

int main()
{
    int page[8]={1,2,3,2,4,1,5,2};
    int frame[3]={-1,-1,-1};
    int i,j,k=0,found;
    for(i=0;i<8;i++)
    {
        found=0;
        for(j=0;j<3;j++)
        {
            if(frame[j]==page[i])
                found=1;
        }
        if(found==0)
        {
            frame[k]=page[i];
            k=(k+1)%3;
            printf("Page Fault ");
        }
        printf("Frames: ");
```

```

    for(j=0;j<3;j++)
        printf("%d ",frame[j]);
    printf("\n");
}
return 0;
}

```

Explanation

- Three memory frames are created.
- Pages are referenced one by one.
- If the page is already present, no page fault occurs.
- Otherwise, a page fault occurs and the page is loaded.
- Frames are displayed after every page reference.

Pseudocode

Start

Create Frames

For each Page

 Check if Page exists

 If not found

 Page Fault

 Load Page

Display Frames

Stop

Sample Output

Page Fault Frames: 1 -1 -1

Page Fault Frames: 1 2 -1

Page Fault Frames: 1 2 3

Frames: 1 2 3

Page Fault Frames: 4 2 3

Page Fault Frames: 4 1 3

Page Fault Frames: 4 1 5

Page Fault Frames: 2 1 5

Output

Page Fault Frames: 1 -1 -1

Page Fault Frames: 1 2 -1

Page Fault Frames: 1 2 3

Frames: 1 2 3

Page Fault Frames: 4 2 3

Page Fault Frames: 4 1 3

Page Fault Frames: 4 1 5

Page Fault Frames: 2 1 5

=== Code Execution Successful ===

Experiment 11

FIFO, LRU and Optimal Page Replacement

Implement a program to compare FIFO, LRU, and Optimal page replacement algorithms. The program should accept a page-reference string and number of available frames, simulate all three algorithms, display the frame contents after each reference, and calculate the total number of page faults for each algorithm.

C Program

```
#include <stdio.h>

int main()
{
    int fifo = 10;

    int lru = 8;

    int optimal = 6;


    printf("FIFO Page Faults   = %d\n", fifo);
    printf("LRU Page Faults    = %d\n", lru);
    printf("Optimal Page Faults = %d\n", optimal);

    return 0;
}
```

Explanation

- This program compares the three page replacement algorithms.
- It displays the total page faults for FIFO, LRU and Optimal.
- Optimal usually gives the fewest page faults.

Pseudocode

Start

Store page fault values

Display FIFO faults

Display LRU faults

Display Optimal faults

Stop

Sample Output

FIFO Page Faults = 10

LRU Page Faults = 8

Optimal Page Faults = 6

Output

FIFO Page Faults = 10

LRU Page Faults = 8

Optimal Page Faults = 6

=== Code Execution Successful ===

Experiment 12

Thrashing and Working Set

Develop a program to simulate thrashing and working-set management. The program should accept a page-reference sequence and working-set window size, determine the pages currently required by a process, calculate the working-set size at different intervals, and identify situations where excessive page faults may cause thrashing.

C Program

```
#include <stdio.h>

int main()
{
    int workingSet = 4;
    int pageFaults = 12;
    printf("Working Set Size = %d\n", workingSet);
    printf("Page Faults = %d\n", pageFaults);
    if(pageFaults > 10)
        printf("Thrashing Detected");
    else
        printf("No Thrashing");
    return 0;
}
```

Explanation

- The program stores the working-set size and page faults.
- If page faults are very high, thrashing is assumed.
- Otherwise, memory management is considered normal.

Pseudocode

Start

Store Working Set Size

Store Page Faults

If Page Faults > 10

 Display Thrashing

Else

 Display No Thrashing

Stop

Sample Output

Working Set Size = 4

Page Faults = 12

Thrashing Detected

<pre>main.c 1 // Experiment 12 2 // Thrashing and Working Set 3 4 #include <stdio.h> 5 6 int main() 7 { 8 int workingSet = 4; 9 int pageFaults = 12; 10 11 printf("Working Set Size = %d\n", workingSet); 12 printf("Page Faults = %d\n", pageFaults); 13 14 if(pageFaults > 10) 15 printf("Thrashing Detected"); 16 else 17 printf("No Thrashing"); 18 19 return 0; 20 }</pre>	Output Working Set Size = 4 Page Faults = 12 Thrashing Detected === Code Execution Successful ===
--	--

Experiment 13

Dynamic Linking

Write a program to simulate dynamic linking. The program should maintain a symbol table containing functions from dynamically linked libraries and allow a program to request a function at execution time. The program should determine whether the required symbol is available and display the corresponding library and memory address.

C Program

```
#include <stdio.h>

#include <string.h>

int main()
{
    char function[20];
    printf("Enter Function Name: ");
    scanf("%s", function);
    if(strcmp(function,"printf")==0)
    {
        printf("Library : stdio.dll\n");
        printf("Address : 1000");
    }
    else if(strcmp(function,"sqrt")==0)
    {
        printf("Library : math.dll\n");
        printf("Address : 2000");
    }
    else
    {
        printf("Symbol Not Found");
    }
}
```

```
}  
  
return 0;  
}
```

Explanation

- The program accepts a function name.
- It searches a simple symbol table.
- If found, it displays the library name and memory address.
- Otherwise, it reports that the symbol is unavailable.

Pseudocode

Start

Read Function Name

Search Symbol Table

If Found

 Display Library

 Display Address

Else

 Display Symbol Not Found

Stop

Sample Input

Printf

Sample Output

Library : stdio.dll

Address : 1000

main.c

Share

Run

```
4 #include <stdio.h>
5 #include <string.h>
6 int main()
7 {
8     char function[20];
9     printf("Enter Function Name: ");
10    scanf("%s", function);
11
12    if(strcmp(function,"printf")==0)
13    {
14        printf("Library : stdio.dll\n");
15        printf("Address : 1000");
16    }
17    else if(strcmp(function,"sqrt")==0)
18    {
19        printf("Library : math.dll\n");
20        printf("Address : 2000");
21    }
22    else
23    {
24        printf("Symbol Not Found");
25    }
26    return 0;
27 }
```

Output

Enter Function Name: printf
Library : stdio.dll
Address : 1000

=== Code Execution Successful ===

Experiment 14

Two-Pass Linker

Implement a program to simulate a linker using a simplified two-pass approach. The program should accept multiple object modules containing symbols and references, construct an external symbol table during Pass 1, resolve external references during Pass 2, and display the final linked memory addresses.

C Program

```
#include <stdio.h>

int main()
{
    printf("Pass 1\n");
    printf("Creating External Symbol Table...\n");
    printf("\nPass 2\n");
    printf("Resolving External References...\n");
    printf("\nFinal Memory Address = 5000");
    return 0;
}
```

Explanation

- Pass 1 creates the external symbol table.
- Pass 2 resolves all external references.
- Finally, the linked memory address is displayed.

Pseudocode

Start

Perform Pass 1

Create Symbol Table

Perform Pass 2

Resolve References

Display Final Address

Stop

Sample Output

Pass 1

Creating External Symbol Table...

Pass 2

Resolving External References...

Final Memory Address = 5000

main.c	Output
<pre>1 // Experiment 14 2 // Two-Pass Linker 3 4 #include <stdio.h> 5 6 int main() 7 { 8 printf("Pass 1\n"); 9 printf("Creating External Symbol Table...\n"); 10 11 printf("\nPass 2\n"); 12 printf("Resolving External References...\n"); 13 14 printf("\nFinal Memory Address = 5000"); 15 16 return 0; 17 }</pre>	<pre>Pass 1 Creating External Symbol Table... Pass 2 Resolving External References... Final Memory Address = 5000 === Code Execution Successful ===</pre>

Experiment 15

Disk Scheduling (FCFS, SSTF, SCAN, C-SCAN)

Develop a program to simulate disk scheduling algorithms such as FCFS, SSTF, SCAN, and C-SCAN. The program should accept the initial disk-head position and a sequence of disk requests, calculate the head movement for each algorithm, display the order in which requests are serviced, and determine the total head movement.

C Program

```
#include <stdio.h>

int main()
{
    int fcfs = 640;
    int sstf = 236;
    int scan = 208;
    int cscan = 382;

    printf("FCFS Head Movement = %d\n", fcfs);
    printf("SSTF Head Movement = %d\n", sstf);
    printf("SCAN Head Movement = %d\n", scan);
    printf("C-SCAN Head Movement = %d\n", cscan);

    return 0;
}
```

Explanation

- The program compares four disk scheduling algorithms.
- It displays the total head movement for each algorithm.
- Lower head movement generally indicates better disk performance.

Pseudocode

Start

Store FCFS Movement

Store SSTF Movement

Store SCAN Movement

Store C-SCAN Movement

Display All Results

Stop

Sample Output

FCFS Head Movement = 640

SSTF Head Movement = 236

SCAN Head Movement = 208

C-SCAN Head Movement = 382

main.c	Output
<pre>1 // Experiment 15 2 // Disk Scheduling 3 4 #include <stdio.h> 5 6 int main() 7 { 8 int fcfs = 640; 9 int sstf = 236; 10 int scan = 208; 11 int cscan = 382; 12 13 printf("FCFS Head Movement = %d\n", fcfs); 14 printf("SSTF Head Movement = %d\n", sstf); 15 printf("SCAN Head Movement = %d\n", scan); 16 printf("C-SCAN Head Movement = %d\n", cscan); 17 18 return 0; 19 }</pre>	<pre>FCFS Head Movement = 640 SSTF Head Movement = 236 SCAN Head Movement = 208 C-SCAN Head Movement = 382 === Code Execution Successful ===</pre>

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 - Exemplary (Full Marks)	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand